step 1

In [37]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
```

In [2]:
```
df = pd.read_csv('project.csv')
```

In [3]:
```
df.shape
```

Out[3]:
```
(235795, 56)
```

In [4]:
```
df.head()
```

Out[4]:

	FILENAME	URL	URLLength	Domain	Do
0	521848.txt	https://www.southbankmosaics.com	31	www.southbankmosaics.com	
1	31372.txt	https://www.uni-mainz.de	23	www.uni-mainz.de	
2	597387.txt	https://www.voicefmradio.co.uk	29	www.voicefmradio.co.uk	
3	554095.txt	https://www.sfnmjournal.com	26	www.sfnmjournal.com	
4	151578.txt	https://www.rewildingargentina.org	33	www.rewildingargentina.org	

5 rows × 56 columns

step 2 :check and clean data

In [5]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 235795 entries, 0 to 235794
Data columns (total 56 columns):
 #   Column                       Non-Null Count   Dtype  
---  ------                       --------------   -----  
 0   FILENAME                     235795 non-null  object 
 1   URL                          235795 non-null  object 
 2   URLLength                    235795 non-null  int64  
 3   Domain                       235795 non-null  object 
 4   DomainLength                 235795 non-null  int64  
 5   IsDomainIP                   235795 non-null  int64  
 6   TLD                          235795 non-null  object 
 7   URLSimilarityIndex           235795 non-null  float64
 8   CharContinuationRate         235795 non-null  float64
 9   TLDLegitimateProb            235795 non-null  float64
 10  URLCharProb                  235795 non-null  float64
 11  TLDLength                    235795 non-null  int64  
 12  NoOfSubDomain                235795 non-null  int64  
 13  HasObfuscation               235795 non-null  int64  
 14  NoOfObfuscatedChar           235795 non-null  int64  
 15  ObfuscationRatio             235795 non-null  float64
 16  NoOfLettersInURL             235795 non-null  int64  
 17  LetterRatioInURL             235795 non-null  float64
 18  NoOfDegitsInURL              235795 non-null  int64  
 19  DegitRatioInURL              235795 non-null  float64
 20  NoOfEqualsInURL              235795 non-null  int64  
 21  NoOfQMarkInURL               235795 non-null  int64  
 22  NoOfAmpersandInURL           235795 non-null  int64  
 23  NoOfOtherSpecialCharsInURL   235795 non-null  int64  
 24  SpacialCharRatioInURL        235795 non-null  float64
 25  IsHTTPS                      235795 non-null  int64  
 26  LineOfCode                   235795 non-null  int64  
 27  LargestLineLength            235795 non-null  int64  
 28  HasTitle                     235795 non-null  int64  
 29  Title                        235795 non-null  object 
 30  DomainTitleMatchScore        235795 non-null  float64
 31  URLTitleMatchScore           235795 non-null  float64
 32  HasFavicon                   235795 non-null  int64  
 33  Robots                       235795 non-null  int64  
 34  IsResponsive                 235795 non-null  int64  
 35  NoOfURLRedirect              235795 non-null  int64  
 36  NoOfSelfRedirect             235795 non-null  int64  
 37  HasDescription               235795 non-null  int64  
 38  NoOfPopup                    235795 non-null  int64  
 39  NoOfiFrame                   235795 non-null  int64  
 40  HasExternalFormSubmit        235795 non-null  int64  
 41  HasSocialNet                 235795 non-null  int64  
 42  HasSubmitButton              235795 non-null  int64  
 43  HasHiddenFields              235795 non-null  int64  
 44  HasPasswordField             235795 non-null  int64  
 45  Bank                         235795 non-null  int64  
 46  Pay                          235795 non-null  int64  
 47  Crypto                       235795 non-null  int64  
 48  HasCopyrightInfo             235795 non-null  int64  
 49  NoOfImage                    235795 non-null  int64  
 50  NoOfCSS                      235795 non-null  int64  
```

```
 51  NoOfJS                     235795 non-null  int64
 52  NoOfSelfRef                235795 non-null  int64
 53  NoOfEmptyRef               235795 non-null  int64
 54  NoOfExternalRef            235795 non-null  int64
 55  label                      235795 non-null  int64
dtypes: float64(10), int64(41), object(5)
memory usage: 100.7+ MB
```

In [6]:
```
df.isnull().sum().sum()
```

Out[6]:
```
np.int64(0)
```

In [7]:
```
df.duplicated().sum()
```

Out[7]:
```
np.int64(0)
```

In [8]:
```
df['label'].value_counts()
```

Out[8]:
```
label
1    134850
0    100945
Name: count, dtype: int64
```

In [9]:
```
df.select_dtypes(include='object').columns
# got 5 object
```

Out[9]:
```
Index(['FILENAME', 'URL', 'Domain', 'TLD', 'Title'], dtype='object')
```

In [10]:
```
df_clean = df.drop(columns=['FILENAME', 'URL', 'Domain', 'Title'])
df_clean.shape
```

Out[10]:
```
(235795, 52)
```

In [11]:
```
df['URL'].nunique()
```

Out[11]:
```
235370
```

In [12]:
```
df['Domain'].nunique()
```

Out[12]:
```
220086
```

In [13]:
```
df['Title'].nunique()
```

Out[13]:
```
197874
```

In [14]:
```
df['TLD'].nunique()
```

Out[14]:
```
695
```

step 3 :Convert the TLD to a number and store it in a new column.

In [15]:
```
le = LabelEncoder()
```

In [16]:
```
df['TLD_encoded'] = le.fit_transform(df['TLD'])
```

In [17]:
```
df[['TLD','TLD_encoded']].head(10)
# Same code name = same number
```

Out[17]:

	TLD	TLD_encoded
0	com	231
1	de	254
2	uk	647
3	com	231
4	org	503
5	org	503
6	com	231
7	com	231
8	in	368
9	org	503

In [18]:
```
df_clean = df_clean.drop(columns=['TLD'])
df_clean.shape
```

Out[18]:
```
(235795, 51)
```

step 4

In [24]:
```
df_clean['TLD_encoded'] = df['TLD_encoded']
df = df_clean.copy()

print(df.columns.tolist())
```

```
['URLLength', 'DomainLength', 'IsDomainIP', 'URLSimilarityIndex', 'CharContinuationRate', 'TLDLegitimateProb', 'URLCharProb', 'TLDLength', 'NoOfSubDomain', 'HasObfuscation', 'NoOfObfuscatedChar', 'ObfuscationRatio', 'NoOfLettersInURL', 'LetterRatioInURL', 'NoOfDegitsInURL', 'DegitRatioInURL', 'NoOfEqualsInURL', 'NoOfQMarkInURL', 'NoOfAmpersandInURL', 'NoOfOtherSpecialCharsInURL', 'SpacialCharRatioInURL', 'IsHTTPS', 'LineOfCode', 'LargestLineLength', 'HasTitle', 'DomainTitleMatchScore', 'URLTitleMatchScore', 'HasFavicon', 'Robots', 'IsResponsive', 'NoOfURLRedirect', 'NoOfSelfRedirect', 'HasDescription', 'NoOfPopup', 'NoOfiFrame', 'HasExternalFormSubmit', 'HasSocialNet', 'HasSubmitButton', 'HasHiddenFields', 'HasPasswordField', 'Bank', 'Pay', 'Crypto', 'HasCopyrightInfo', 'NoOfImage', 'NoOfCSS', 'NoOfJS', 'NoOfSelfRef', 'NoOfEmptyRef', 'NoOfExternalRef', 'label', 'TLD_encoded']
```

In [34]:
```
# before drop label is 52

X = df.drop(columns=['label'])
y = df['label']
# after is 51
# A row that only displays a label.
print("X shape:", X.shape)
print("y shape", y.shape)
```

```
X shape: (235795, 51)
y shape (235795,)
```

step 5 Train/Test Split

In [35]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# stratify=y  (Ensure that the training set and the test set have the same number o

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (188636, 51)
X_test shape: (47159, 51)
y_train shape: (188636,)
y_test shape: (47159,)
```

In [26]:
```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Done scaling")
print(X_train_scaled[:2])
```

```
Done scaling
[[-0.2582902  -0.48816772 -0.05287945  0.74445584  0.71408399  1.04458853
   0.61394113  0.39167177 -0.27544102 -0.04522324 -0.01283941 -0.03592044
  -0.30284789 -0.4687109  -0.1502932  -0.40442994 -0.06329513 -0.15228304
  -0.02886874 -0.36535123 -0.65941076  0.52699844  0.01628539 -0.05668231
   0.4015167   1.00475223  0.96647456  1.32776742 -0.60317853  0.77482883
  -0.39283965 -0.20346893  1.12791966 -0.05581924 -0.26949991 -0.21480507
   1.0914676   1.18748865  1.28403645 -0.33760122 -0.38134147 -0.55809124
  -0.15484475  1.02575621  0.14537643  0.17510447  0.57329527  0.54386186
  -0.07533932 -0.14896875 -0.61281228]
 [-0.30709745 -0.70633344 -0.05287945  0.74445584 -0.43902575 -0.94206582
   1.08837153 -1.27293105  1.38620764 -0.04522324 -0.01283941 -0.03592044
  -0.41074082 -1.23110344 -0.1502932  -0.40442994 -0.06329513 -0.15228304
  -0.02886874 -0.09389713  0.85382815  0.52699844  0.21173693 -0.06345621
   0.4015167  -1.00837914 -1.04964484  1.32776742 -0.60317853  0.77482883
  -0.39283965 -0.20346893  1.12791966  0.19248917  0.07114762 -0.21480507
   1.0914676  -0.84211331 -0.77879409 -0.33760122 -0.38134147 -0.55809124
  -0.15484475  1.02575621  0.72254211 -0.04047034 -0.14987336 -0.02424986
   0.08687689 -0.19164467  0.46405969]]
```

step 6

In [27]:
```python
# Building the Model
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)

# Train the model (using scaled training data)
rf_model.fit(X_train_scaled, y_train)

print("Training done!")
```

```
Training done!
```

In [36]:
```python
# Predict using test data

# Work out the answers yourself for the questions you haven't looked at yet.
y_pred = rf_model.predict(X_test_scaled)

# Print the 4 metrics
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

```
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
```

In [29]:
```python
# Get feature importance from the trained model
importance = pd.DataFrame({
    'feature': X.columns,
    'importance': rf_model.feature_importances_
})

# Sort by importance, highest first
```

```
importance = importance.sort_values('importance', ascending=False)

print(importance.head(10))
```

```
                feature  importance
3    URLSimilarityIndex    0.190299
49      NoOfExternalRef    0.127629
47          NoOfSelfRef    0.108896
46               NoOfJS    0.108858
22           LineOfCode    0.102539
44            NoOfImage    0.083501
45              NoOfCSS    0.051608
36        HasSocialNet    0.050915
43     HasCopyrightInfo    0.034942
32       HasDescription    0.025167
```

In [32]:
```
# Try training WITHOUT the most dominant feature
X_test_v2 = X.drop(columns=['URLSimilarityIndex'])

X_train2, X_test2, y_train2, y_test2 = train_test_split(
    X_test_v2, y, test_size=0.2, random_state=42, stratify=y
)

scaler2 = StandardScaler()
X_train2_scaled = scaler2.fit_transform(X_train2)
X_test2_scaled = scaler2.transform(X_test2)

rf_model2 = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model2.fit(X_train2_scaled, y_train2)

y_pred2 = rf_model2.predict(X_test2_scaled)

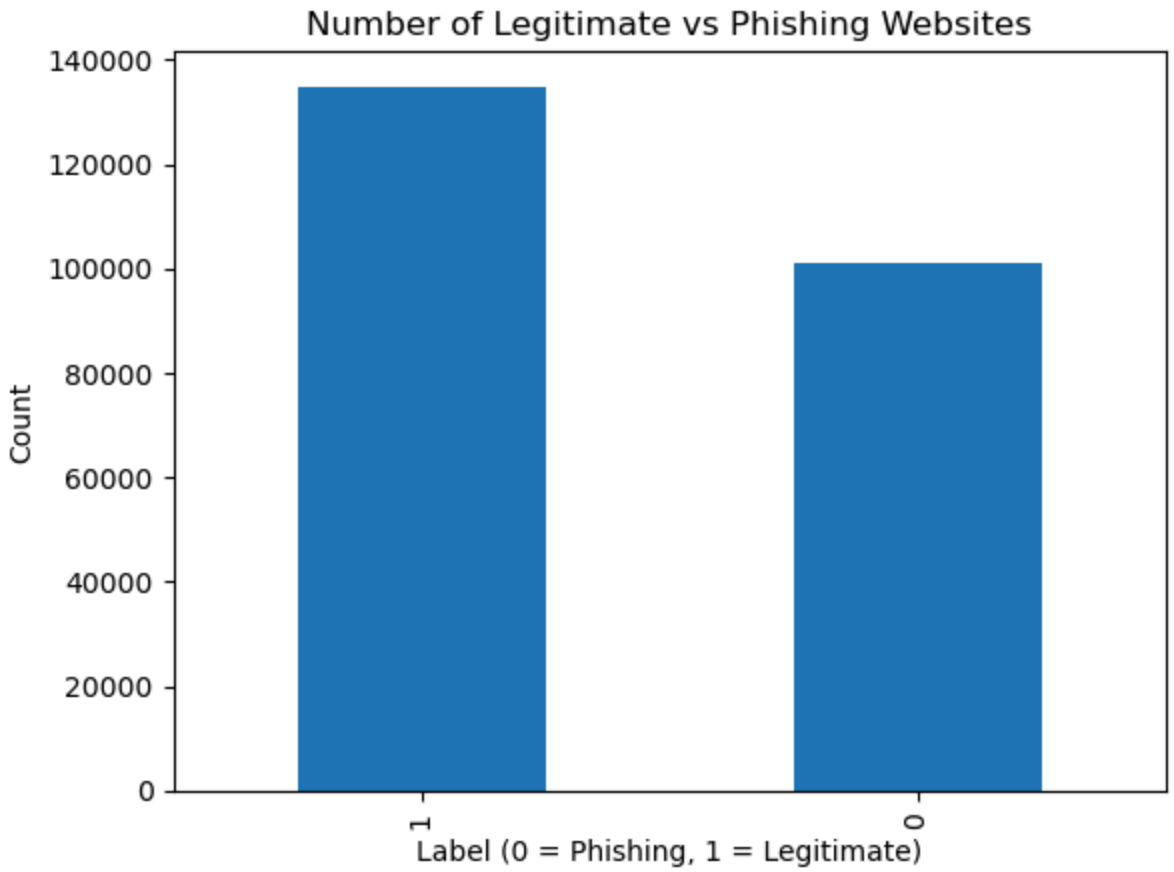
print("Accuracy without URLSimilarityIndex:", accuracy_score(y_test2, y_pred2))
```

```
Accuracy without URLSimilarityIndex: 0.9998939756992302
```

In [ ]:
```
# We tested removing the most dominant feature (URLSimilarityIndex)
# and retrained the model. Accuracy remained at 99.99%, confirming that the
# near-perfect performance is not due to a single leaking feature, but rather
# the highly engineered nature of this dataset's features (as also observed
# in published research using this dataset).
```

Step 7 EDA

In [38]:
```
df['label'].value_counts().plot(kind='bar')
plt.title('Number of Legitimate vs Phishing Websites')
plt.xlabel('Label (0 = Phishing, 1 = Legitimate)')
plt.ylabel('Count')
plt.show()
```



In [40]:
```
# Use percentile to filter out extreme outliers just for visualization
for col in ['URLLength', 'LineOfCode', 'NoOfImage', 'NoOfJS']:
    upper_limit = df[col].quantile(0.95)  # look at the bottom 95% of data
    print(col, "95th percentile value:", upper_limit)
```

```
URLLength 95th percentile value: 73.0
LineOfCode 95th percentile value: 4391.0
NoOfImage 95th percentile value: 96.0
NoOfJS 95th percentile value: 38.0
```

In [41]:
```
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

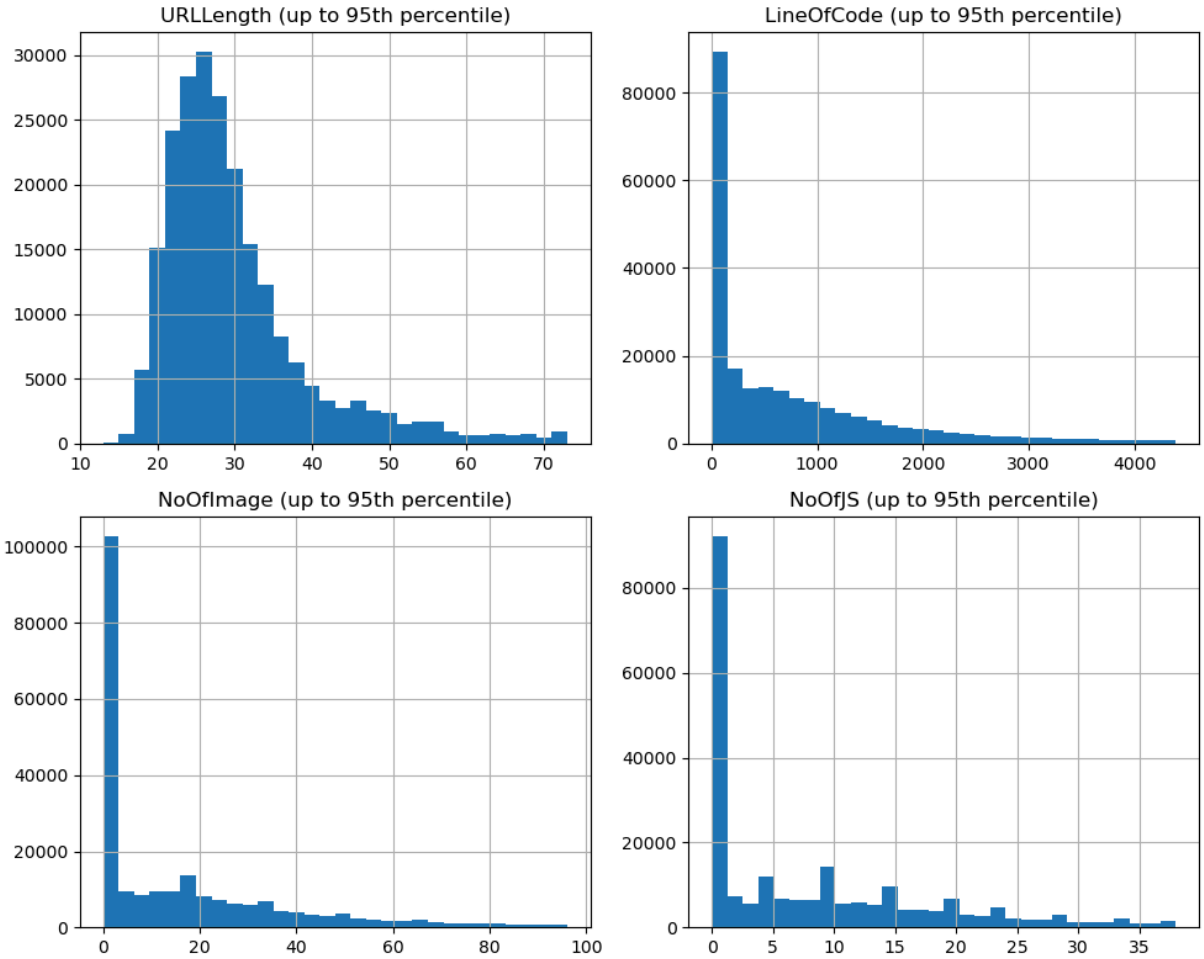
df[df['URLLength'] <= 73]['URLLength'].hist(bins=30, ax=axes[0, 0])
axes[0, 0].set_title('URLLength (up to 95th percentile)')

df[df['LineOfCode'] <= 4391]['LineOfCode'].hist(bins=30, ax=axes[0, 1])
axes[0, 1].set_title('LineOfCode (up to 95th percentile)')

df[df['NoOfImage'] <= 96]['NoOfImage'].hist(bins=30, ax=axes[1, 0])
axes[1, 0].set_title('NoOfImage (up to 95th percentile)')

df[df['NoOfJS'] <= 38]['NoOfJS'].hist(bins=30, ax=axes[1, 1])
axes[1, 1].set_title('NoOfJS (up to 95th percentile)')

plt.tight_layout()
plt.show()
```


URLLength (up to 95th percentile)
LineOfCode (up to 95th percentile)
NoOfImage (up to 95th percentile)
NoOfJS (up to 95th percentile)

In [ ]: